



Agentic Loops

วงจรการทำงานหลักของ Claude Agent

Domain 1 · Task 1.1 — Claude Certified Architect

Agentic Loop คืออะไร?

- คือ **วงจรการทำงาน (execution cycle)** ที่ขับเคลื่อน agent ทุกตัวที่สร้างบน Claude
- เป็นรูปแบบ **control flow** ที่มีขั้นตอนชัดเจน ตายตัว **คาดเดาผลได้**

× ไม่ใช่

เทคนิคการเขียน prompt

× ไม่ใช่

retry loop ธรรมดา

× ไม่ใช่

แค่การตอบกลับของ chatbot

เป็น **พื้นฐานที่ต้องเข้าใจ** ทั้งสำหรับสอบและระบบ production

วงจร 4 ขั้นตอน

- 1 ส่ง request ไปที่ Claude ผ่าน **Messages API** (แบบประวัติสนทนาทั้งหมด รวม tool results รอบก่อน)
- 2 ตรวจสอบฟิลด์ `stop_reason` ในคำตอบ — **สัญญาณหลัก**ที่บอกว่าต้องทำอะไรต่อ
- 3 ถ้าเป็น `"tool_use"` → รับ tool ที่ Claude ขอ แล้ว **append** ผลกลับเข้า history → ส่งกลับไปใหม่
- 4 ถ้าเป็น `"end_turn"` → **งานเสร็จ** ส่งคำตอบสุดท้ายให้ผู้ใช้

ขั้นตอน 3 จะกลับไปขั้นตอน 1 จนกว่าจะถึง `end_turn` ↺

จุดสำคัญที่ห้ามพลาด

⚠ **Tool results ต้อง append เสมอ**

ต้องใส่ผลลัพธ์ของ tool กลับเข้า conversation history ทุกครั้ง

ถ้าไม่ทำ → Claude มองไม่เห็นผลลัพธ์ และตัดสินใจรอบถัดไปไม่ได้

🔑 **Key Concept: stop_reason**

คือสัญญาณเดียวที่เชื่อถือได้สำหรับควบคุม loop

ให้ค่าที่ชัดเจน ตีความได้ทางเดียว ไม่กำกวม

🚫 ห้ามใช้เป็นกลไกหลัก: parse ข้อความ · เช็ค text content · จำนวนรอบตายตัว

ค่า stop_reason ในโลกจริง

📖 ที่ข้อสอบเน้น (v0.2)

tool_use

end_turn

🌐 API จริงมีเพิ่ม (production ต้องรองรับ)

pause_turn

max_tokens

stop_sequence

refusal

💡 หลักคิด

ถ้าไม่ใช่ end_turn → ถือว่า "ยังไม่จบ ต้องเช็คสาเหตุ"

ไม่ใช่เดาว่าเป็น tool_use

Model-Driven Decision-Making

✓ Model-driven

ตัวโมเดล**เลือกเอง**ว่าจะเรียก tool ไหน ตามบริบท

vs. Developer-driven

decision tree / fixed sequence ที่เขียนตายตัว

🔪 ข้อสอบให้น้ำหนักกับ model-driven — **ยึดหยุ่น** รับมือ edge case ได้ดีกว่า

⚡ ข้อยกเว้น

งานที่ผลลัพธ์ต้อง**ถูกต้อง 100% ทุกครั้ง**

การเงิน

security

ข้อกำกับดูแล

→ ต้องบังคับด้วย**โค้ด (programmatic enforcement)** แทน
การปล่อยให้โมเดลตัดสินใจ

รายละเอียดใน Task 1.4

3 Anti-Patterns ของการหยุด loop

★ ออกสอบบ่อย

1. Parse ข้อความ

เชื่อว่า Claude พูดว่า "เสร็จแล้ว" — **ภาษาธรรมชาติกำกวมเสมอ**

2. ใช้ iteration cap เป็นกลไกหลัก

เช่น "หยุดหลัง 10 รอบ" → ตัดงานที่ยังไม่เสร็จ หรือวนเกินจำเป็น

(ใช้เป็น safety net ได้ แต่ห้ามเป็นกลไกหลัก)

3. เช็ค content type

เช่น `response.content[0].type == "text"`

Claude ส่ง text พร้อมกับ tool_use block ในคำตอบเดียวกันได้ — มี text ≠ จบงาน



Case Study: Premature Termination

อาการ

Agent ฝ่าย support ทำงานได้กับคำถามง่าย แต่**หยุดกลางคัน**กับงานซับซ้อน

สาเหตุ

โค้ดเช็ค `content[0].type == "text"`

แต่ Claude ตอบ text อธิบาย ("*ขอใช้คอปเตอร์ก่อนนะ*") **พร้อม** tool_use block

→ โค้ดเห็น text เลยคิดว่า**จบงาน** ทั้งที่ยังต้องเรียก tool ต่อ

✅ วิธีแก้

เปลี่ยนมาเช็ค `stop_reason` — วนต่อเมื่อ `tool_use` · หยุดเมื่อ `end_turn`

! Exam Traps ที่ต้องระวัง

ตัวหลอก #1 — "เพิ่ม iteration cap"

ข้อสอบเสนอเป็นคำตอบหลอกสำหรับปัญหา premature termination

× ตัดทิ้ง — cap แก้ปัญหา loop วนไม่หยุด **ไม่ได้** แก้ปัญหาหยุดก่อนเวลา

ตัวหลอก #2 — บังคับ `tool_choice = "any"`

เพื่อกัน agent ตอบเป็น text

× ผล: บังคับให้เรียก tool แม้งานเสร็จแล้ว → **infinite loop**

✅ คำตอบที่ถูกเสมอ

ให้โมเดลส่งสัญญาณจบผ่าน `stop_reason` **ตามธรรมชาติ**

สรุป Task 1.1



Agentic loop = execution cycle 4 ชั้น



`stop_reason` = สัญญาณควบคุม loop เดียวที่เชื่อถือได้



Append tool results เข้า history เสมอ



Model-driven > fixed sequence (ยกเว้น งาน 100%)



ระวัง trap: iteration cap ·
`tool_choice="any"`



อย่าเช็ค content type เป็นเงื่อนไขจบงาน



Exam Sim — ข้อ 1/5 (ลองคิดก่อนดูเฉลย)

agent ของนักพัฒนาคนหนึ่งบางครั้ง**หยุดทำงานก่อนเวลา**เมื่อ Claude ตอบ text มาพร้อมกับ tool call — โค้ด loop ของเขาเช็ค `response.content[0].type == "text"` เพื่อดูว่า agent ทำงานเสร็จหรือยัง ผู้ใช้รายงานว่าได้คำตอบ ไม่ครบเมื่อถามคำถามซ้ำซ้อน นักพัฒนาควรเปลี่ยนอะไร?

- A. เพิ่ม iteration cap เป็น 15 รอบ เพื่อให้ agent วนนานพอสำหรับคำถามซ้ำซ้อน
- B. เช็คฟิลด์ stop_reason แทนการเช็ค content type — วนต่อเมื่อ stop_reason เป็น "tool_use" และหยุดเมื่อเป็น "end_turn"
- C. parse ข้อความของ assistant หาสีบอกรอบงานอย่าง "I have finished" ก่อนจะหยุด loop
- D. ตั้ง tool_choice เป็น "any" เพื่อให้ Claude เรียก tool เสมอแทนที่จะตอบเป็น text

⏸ หยุดคิดสัก 30 วินาที แล้วค่อยไปดูเฉลยหน้าถัดไป



เฉลย Exam Sim — ข้อ 1/5

คำตอบที่ถูก: Option B

stop_reason คือสัญญาณที่ deterministic และเชื่อถือได้ที่สุดในการควบคุม loop มันแยกแยะได้ชัดว่า Claude อยากเรียก tool ต่อ (tool_use) หรือทำงานเสร็จแล้ว (end_turn) โดยไม่สนว่าจะมี text ปนมากับ tool call หรือไม่

ทำไมข้ออื่นผิด

- A — cap แบบตายตัวไม่แก้ต้นเหตุ agent หยุดเพราะอ่านชนิดคำตอบผิด ไม่ใช่เพราะวนน้อยไป cap 15 ก็ยังหยุดก่อนเวลาถ้า bug เช็ค text ทำงานตั้งแต่รอบ 2
- C — การ parse ภาษาธรรมชาติกำกวมและไม่น่าเชื่อถือ Claude อาจบอกว่าเสร็จขึ้นหนึ่งแต่ตั้งใจทำต่อ ขณะที่ stop_reason ให้สัญญาณชัดเจนอยู่แล้ว
- D — บังคับเรียก tool แม้งานเสร็จจริงจะกลายเป็น infinite loop ปัญหาไม่ใช่ที่ Claude ตอบ text แต่คือโค้ดตีความว่ามี text = จบงาน



Exam Sim — ข้อ 2/5 (ลองคิดก่อนดูเฉลย)

ข้อใดอธิบาย lifecycle ของ agentic loop ได้ถูกต้อง?

- A. ส่ง request, เช็คว่าคำตอบมี text ไหม, parse text หาคำสั่ง, รับ tool ที่ถูกกล่าวถึงใน text
- B. ส่ง request, ตรวจสอบ stop_reason, ถ้าเป็น "tool_use" รับ tool แล้ว append ผลเข้า history, ถ้าเป็น "end_turn" จบงาน
- C. ส่ง request, รับ tool ทุกตัวที่มีเรียงตามลำดับ, เช็คว่ามี tool ไหน error, retry tool ที่ล้มเหลว
- D. ส่ง request, นับ token ในคำตอบ, ถ้า token เกิน threshold แปลว่าต้องใช้ tool ถ้าไม่เกินก็จบงาน



หยุดคิดสัก 30 วินาที แล้วค่อยไปดูเฉลยหน้าถัดไป



เฉลย Exam Sim — ข้อ 2/5

คำตอบที่ถูก: Option B

lifecycle ของ agentic loop มี 4 ขั้นตอน deterministic: ส่ง request ผ่าน Messages API, ตรวจสอบ stop_reason, ถ้าเป็น tool_use ให้รับ tool แล้ว append ผลกลับเข้า history, ถ้าเป็น end_turn ก็จบ

ทำไมข้ออื่นผิด

- A — ฟังก์ชันการ parse ภาษาธรรมชาติหาคำสั่งซึ่งกำกวม ตัวที่กำหนดว่าจะทำอะไรต่อไปคือ stop_reason ไม่ใช่ text content
- C — agentic loop ไม่ได้รับ tool ทุกตัวเรียงกัน Claude เลือกเองว่าจะเรียก tool ไหนตามบริบท (model-driven) และไม่ได้ retry อัตโนมัติ
- D — จำนวน token ไม่เกี่ยวกับการควบคุม loop เลย stop_reason เป็นสัญญาณเดียวที่บอกว่าจะวนต่อหรือหยุด



Exam Sim — ข้อ 3/5 (ลองคิดก่อนดูเฉลย)

agent ตัวหนึ่งทำงานง่าย ๆ ได้ถูกต้อง แต่เข้าสู่ **infinite loop** เมื่อเจอคำถามซับซ้อน นักพัฒนาแก้ด้วยการเพิ่ม iteration cap เป็น 10 อะไรคือปัญหาของวิธีนี้?

A. iteration cap ต่ำไป — ควรตั้งอย่างน้อย 50 สำหรับคำถามซับซ้อน

B. iteration cap ใช้เป็น safety net ได้ แต่ไม่ควรเป็นกลไกหยุดหลัก นักพัฒนาควรหาสาเหตุว่าทำไม stop_reason ถึงไม่ถูกเช็คให้ถูกต้อง

C. agent ต้องการ tool เพิ่มเพื่อรับมือคำถามซับซ้อน ไม่ใช่ iteration cap

D. iteration cap ใช้ได้เฉพาะกับ synchronous API เท่านั้น ไม่ได้กับ streaming response

⏸ หยุดคิดสัก 30 วินาที แล้วค่อยไปดูเฉลยหน้าถัดไป



เฉลย Exam Sim — ข้อ 3/5

คำตอบที่ถูก: Option B

การเกิด infinite loop แปลว่า stop_reason ไม่ได้ถูกใช้เป็นกลไกควบคุมหลักอย่างถูกต้อง iteration cap แค่กลบ bug ดันเหตุไว้ ไม่ได้แก้ cap ควรใช้เป็นเพียง safety net กัน agent วิ่งหลุด ไม่ใช่กลไกหลักในการควบคุม loop

ทำไมข้ออื่นผิด

- A — เพิ่มค่า cap ไม่ได้แก้ต้นเหตุ ถ้า loop วนไม่หยุดเพราะไม่เช็ค stop_reason ไม่ว่าตั้ง cap เท่าไรก็ไม่แก้ปัญหา แค่เลื่อนเวลาหยุดออกไป
- C — จำนวน tool ไม่เกี่ยวกับพฤติกรรม infinite loop loop จะวนต่อหรือหยุดขึ้นกับ stop_reason ไม่ใช่ว่ามี tool ที่เหมาะสมหรือไม่
- D — iteration cap ใช้ได้กับทุกโหมดการทำงาน ปัญหาคือการใช้มันเป็นกลไกหลักแทนการเช็ค stop_reason



Exam Sim — ข้อ 4/5 (ลองคิดก่อนดูเฉลย)

ทำไมต้อง append tool results เข้า conversation history ก่อนส่ง request ถัดไปให้ Claude?

- A. เพื่อลดค่า API ด้วยการ cache คำตอบก่อนหน้า
- B. เพื่อให้ Claude ใช้เหตุผลกับข้อมูลใหม่จาก tool ในรอบถัดไป และตัดสินใจว่าจะทำอะไรต่อ
- C. เพื่อให้ stream ผลลัพธ์บางส่วนไปยัง user interface ได้
- D. เพราะ Messages API จะ reject request ที่ไม่มี conversation history ครบถ้วน



หยุดคิดสัก 30 วินาที แล้วค่อยไปดูเฉลยหน้าถัดไป

✓ เฉลย Exam Sim — ข้อ 4/5

คำตอบที่ถูก: Option B

โมเดลต้องเห็นว่า tool ค้นอะไรมา จึงจะตัดสินใจ action ถัดไปได้ ถ้าไม่มี tool results ใน history Claude ก็นำผลลัพธ์ tool มาต่อยอดในสายการให้เหตุผลไม่ได้ และตัดสินใจไม่ได้ว่าจะเรียก tool อีกหรือจบงาน

ทำไมข้ออื่นผิด

- A — การ append tool results เป็นเรื่องความต่อเนื่องของการให้เหตุผล ไม่ใช่การลดค่าใช้จ่าย โมเดลต้องการข้อมูลเพื่อคิด ไม่ใช่เพื่อประหยัดเงิน
- C — การ append tool results เกี่ยวกับสายการให้เหตุผลของโมเดล ไม่ใช่การ stream ผลบางส่วนให้ผู้ใช้ เป็นคนละเรื่องกัน
- D — API ไม่ได้ reject history ที่ไม่ครบเป็นข้อจำกัดทางเทคนิค ประเด็นคือถ้าไม่มี tool results โมเดลจะขาดข้อมูลที่จำเป็นต่อการให้เหตุผลที่ถูกต้อง



Exam Sim — ข้อ 5/5 (ลองคิดก่อนดูเฉลย)

agent ฝ่าย customer support ใช้ model-driven decision-making ในการเลือก tool ในสถานการณ์ใดที่ควรเปลี่ยนมาใช้ programmatic enforcement แทน?

- A. เมื่อ agent จัดการบทสนทนาพร้อมกันมากกว่า 3 บทสนทนา
- B. เมื่องานต้องการความถูกต้องแบบ deterministic สำหรับงานการเงิน, security หรือ regulatory
- C. เมื่อผู้ใช้ร้องขอ tool ตัวใดตัวหนึ่งโดยระบุชื่อในข้อความ
- D. เมื่อ agent เข้าถึง tool ได้มากกว่า 5 ตัว

⏸ หยุดคิดสัก 30 วินาที แล้วค่อยไปดูเฉลยหน้าถัดไป

✓ เฉลย Exam Sim — ข้อ 5/5

คำตอบที่ถูก: Option B

model-driven decision-making เป็นแบบ probabilistic สำหรับงานที่ความล้มเหลวเพียงครั้งเดียวทำให้เกิดความเสียหายทางการเงิน security breach หรือละเมิด compliance การใช้ programmatic enforcement ให้การรับประกันแบบ deterministic ที่เหนือกว่าความยืดหยุ่นของโมเดล

ทำไมข้ออื่นผิด

- A — จำนวนบทสนทนาพร้อมกันไม่ได้กำหนดว่าจะใช้ model-driven หรือ programmatic ตัวที่กำหนดคือความสำคัญ (stakes) ของงาน
- C — คำขอของผู้ใช้ไม่ได้ override การตัดสินใจเชิงสถาปัตยกรรมระหว่าง model-driven กับ programmatic ความสำคัญของงานต่างหากที่กำหนดวิธี
- D — จำนวน tool ที่มีไม่เกี่ยวกับการตัดสินใจเรื่อง enforcement ไม่ว่ามี 2 หรือ 20 tool งาน high-stakes ก็ต้องใช้ programmatic enforcement